

Next JS Assignment 5

Q1 Create a json file with 5 records of teachers and 30 records of students.

```
app > {} data.json > ...
1  {
2    "teachers": [
3      { "id": 1, "name": "Rahul Sharma", "subject": "Math" },
4      { "id": 2, "name": "Priya Verma", "subject": "Science" },
5      { "id": 3, "name": "Amit Singh", "subject": "English" },
6      { "id": 4, "name": "Neha Gupta", "subject": "History" },
7      { "id": 5, "name": "Vikram Joshi", "subject": "Computer" }
8    ],
9    "students": [
10     { "id": 1, "name": "Student1", "teacherId": 1 },
11     { "id": 2, "name": "Student2", "teacherId": 1 },
12     { "id": 3, "name": "Student3", "teacherId": 1 },
13     { "id": 4, "name": "Student4", "teacherId": 1 },
14     { "id": 5, "name": "Student5", "teacherId": 1 },
15
16     { "id": 6, "name": "Student6", "teacherId": 2 },
17     { "id": 7, "name": "Student7", "teacherId": 2 },
18     { "id": 8, "name": "Student8", "teacherId": 2 },
19     { "id": 9, "name": "Student9", "teacherId": 2 },
20     { "id": 10, "name": "Student10", "teacherId": 2 },
21
22     { "id": 11, "name": "Student11", "teacherId": 3 },
23     { "id": 12, "name": "Student12", "teacherId": 3 },
24     { "id": 13, "name": "Student13", "teacherId": 3 },
25     { "id": 14, "name": "Student14", "teacherId": 3 },
26     { "id": 15, "name": "Student15", "teacherId": 3 },
27
28     { "id": 16, "name": "Student16", "teacherId": 4 },
29     { "id": 17, "name": "Student17", "teacherId": 4 },
30     { "id": 18, "name": "Student18", "teacherId": 4 },
31     { "id": 19, "name": "Student19", "teacherId": 4 },
32     { "id": 20, "name": "Student20", "teacherId": 4 },
33
34     { "id": 21, "name": "Student21", "teacherId": 5 },
35     { "id": 22, "name": "Student22", "teacherId": 5 },
36     { "id": 23, "name": "Student23", "teacherId": 5 },
37     { "id": 24, "name": "Student24", "teacherId": 5 },
38     { "id": 25, "name": "Student25", "teacherId": 5 },
39     { "id": 26, "name": "Student26", "teacherId": 5 },
40     { "id": 27, "name": "Student27", "teacherId": 5 },
```

Q2 Create an API endpoint to fetch the list of all the teachers.

The screenshot shows the Postman interface for a workspace named "Aman Mittal's Workspace". The left sidebar contains navigation options: Collections, Environments, History, and Flows. The "Collections" section is active, showing a collection named "My first collection" with two folders: "First folder inside collection" and "Second folder inside collection". Each folder contains a GET request. Below the collection, there is a "Create a collection for your requests" section with a "Create Collection" button.

The main area displays a GET request to the endpoint `http://localhost:3000/api/teachers/`. The request is saved and has a "Send" button. The "Params" tab is selected, showing a table with columns "Key", "Value", "Description", and "Bulk Edit". The "Body" tab is also selected, showing a JSON response with a status of "200 OK", a response time of "21 ms", and a size of "499 B".

The JSON response is as follows:

```
[
  {
    "id": 1,
    "name": "Rahul Sharma",
    "subject": "Math"
  },
  {
    "id": 2,
    "name": "Priya Verma",
    "subject": "Science"
  },
  {
    "id": 3,
    "name": "Amit Singh",
    "subject": "English"
  },
  {
    "id": 4,
    "name": "Neha Gupta",
    "subject": "History"
  },
  {
    "id": 5,
    "name": "Vikram Joshi",
    "subject": "Computer"
  }
]
```

Q3 Create an API endpoint to fetch the list of all the students.

The screenshot displays the Postman interface for a workspace named "Aman Mittal's Workspace". The main panel shows a GET request to the endpoint `http://localhost:3000/api/students/` which has been successfully executed, returning a 200 OK status with a response time of 22 ms and a body size of 1.49 KB. The response body is a JSON array containing 7 student records.

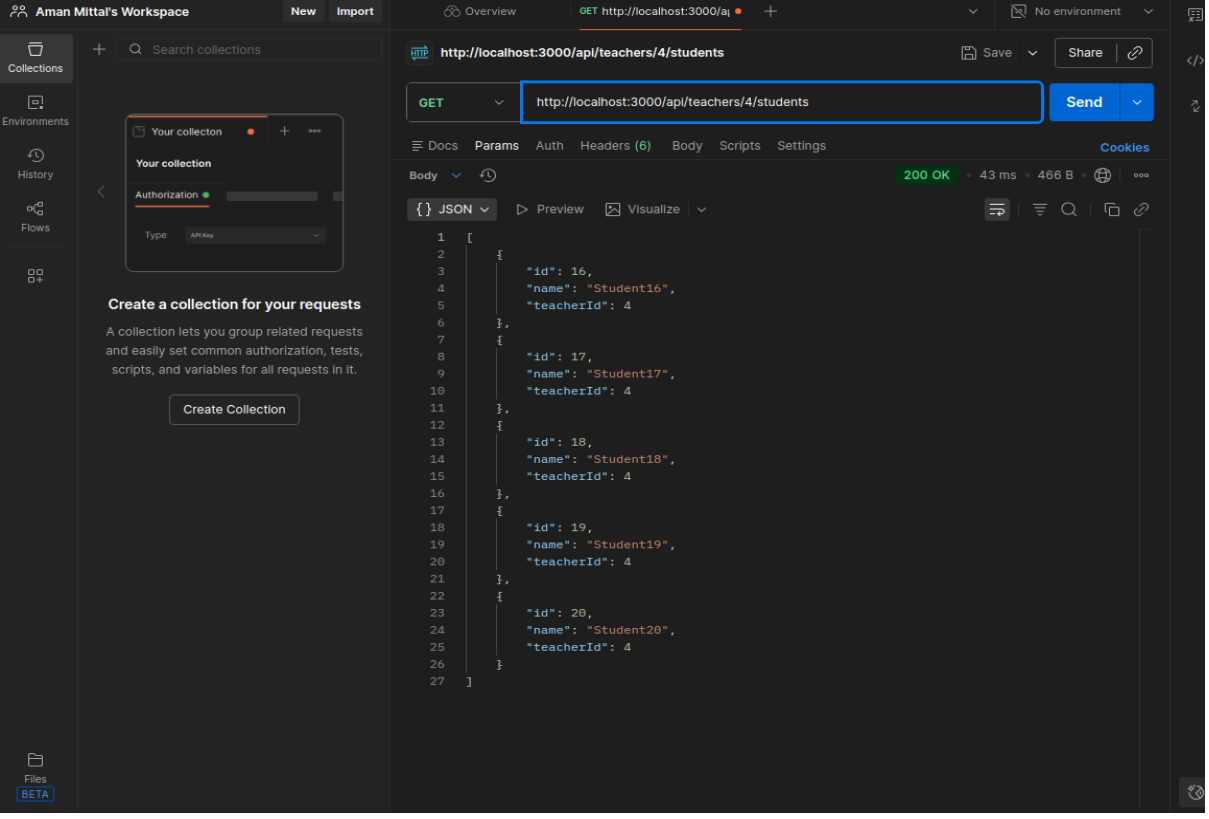
Request Details:

- Method: GET
- URL: `http://localhost:3000/api/students/`

Response Body (JSON):

```
[
  {
    "id": 1,
    "name": "Student1",
    "teacherId": 1
  },
  {
    "id": 2,
    "name": "Student2",
    "teacherId": 1
  },
  {
    "id": 3,
    "name": "Student3",
    "teacherId": 1
  },
  {
    "id": 4,
    "name": "Student4",
    "teacherId": 1
  },
  {
    "id": 5,
    "name": "Student5",
    "teacherId": 1
  },
  {
    "id": 6,
    "name": "Student6",
    "teacherId": 2
  },
  {
    "id": 7,
    "name": "Student7",
    "teacherId": 2
  }
]
```

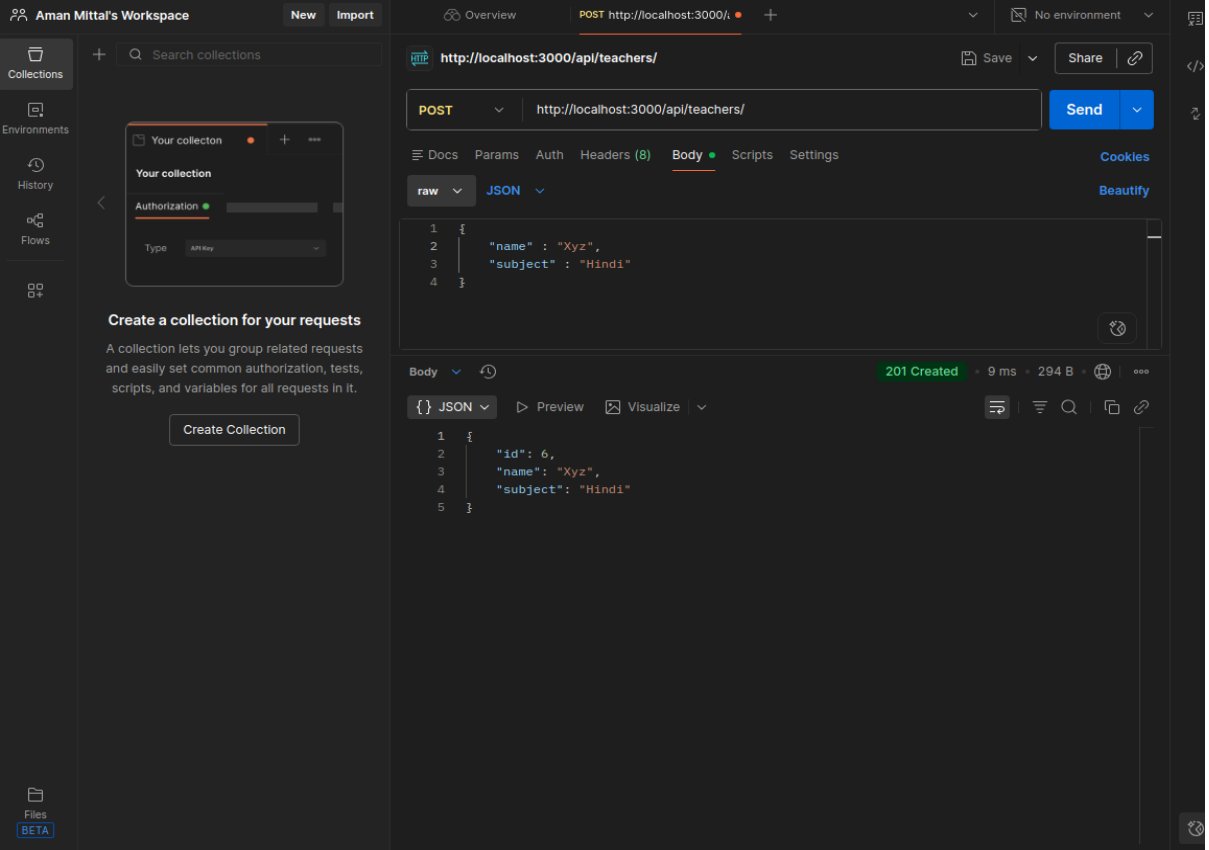
Q4 Create an API endpoint to fetch the list of students who are under a specific teacher.



The screenshot shows the Postman interface with a GET request to `http://localhost:3000/api/teachers/4/students`. The response is a 200 OK status with a JSON array of 5 student objects. The JSON body is as follows:

```
1 [
2   {
3     "id": 16,
4     "name": "Student16",
5     "teacherId": 4
6   },
7   {
8     "id": 17,
9     "name": "Student17",
10    "teacherId": 4
11  },
12  {
13    "id": 18,
14    "name": "Student18",
15    "teacherId": 4
16  },
17  {
18    "id": 19,
19    "name": "Student19",
20    "teacherId": 4
21  },
22  {
23    "id": 20,
24    "name": "Student20",
25    "teacherId": 4
26  }
27 ]
```

Q5 Create an endpoint to add a teacher in the list.



The screenshot shows the Postman interface with a POST request to `http://localhost:3000/api/teachers/`. The request body is a JSON object with `"name": "Xyz"` and `"subject": "Hindi"`. The response is a 201 Created status with a JSON object containing `"id": 6`, `"name": "Xyz"`, and `"subject": "Hindi"`.

```
1 {
2   "name": "Xyz",
3   "subject": "Hindi"
4 }

Body

1 {
2   "id": 6,
3   "name": "Xyz",
4   "subject": "Hindi"
5 }
```